Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет: Инженерно-экономический

Кафедра: Экономической информатики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
на тему:
«Технология Java TM Remote Method Invocation (Java RMI)»

Выполнил Летко А. С.
гр. 377901

Проверил Русецкий А.Д.

Минск 2026

Цель работы

Целью данной лабораторной работы является изучение принципов построения распределенных Java-приложений с использованием технологии Java RMI (Remote Method Invocation).

В ходе выполнения работы необходимо освоить:

— понятие удаленного объекта и его роль в распределенных системах;

— механизм удаленного вызова методов между различными виртуальными машинами Java;

— архитектуру клиент-серверного взаимодействия на основе RMI;

— работу с RMI-регистратором (RMI Registry) как службой имен;

— обработку исключений в распределенных системах;

— реализацию как синхронного, так и асинхронного (с использованием callback) удаленного вызова.

Задачи работы

Для достижения поставленной цели были определены следующие задачи:

1 Изучить теоретические основы технологии Java RMI, включая:

— различия между RMI и RPC;

— архитектуру RMI-приложения;

— компоненты RMI (удаленный интерфейс, stub, серверная реализация, RMI Registry).

2 Разработать распределенное приложение, реализующее вариант задания №8 — вычисление наименьшего общего кратного (НОК) двух целых чисел.

3 Реализовать синхронную версию приложения, при которой клиент блокируется до получения результата.

4 Реализовать асинхронную версию приложения с использованием callback-механизма, при которой клиент не блокируется, а результат приходит позже через обратный вызов.

5 Организовать обработку нескольких клиентов с помощью пула потоков на сервере.

6 Реализовать дружественный интерфейс клиента с обработкой всех возможных ошибок ввода.

7 Протестировать работу приложения и проанализировать результаты.

Теоретическая часть

Понятие распределенных систем

Распределенные системы представляют собой совокупность независимых компьютеров, которые взаимодействуют между собой для решения общей задачи. В таких системах объекты вычислений могут находиться в различных адресных пространствах и на различных физических узлах. Для их взаимодействия необходимы механизмы коммуникации.

RPC и RMI: сравнительный анализ

RPC (Remote Procedure Call) — технология удаленного вызова процедур, позволяющая вызывать функции на удаленном компьютере так, как если бы они были локальными. RPC является процедурно-ориентированной технологией.

RMI (Remote Method Invocation) — объектно-ориентированная технология удаленного вызова методов, разработанная специально для платформы Java.

Архитектура RMI-приложения

Типичное RMI-приложение состоит из следующих компонентов:

1 Удаленный интерфейс (Remote Interface) — определяет контракт между клиентом и сервером. Должен расширять интерфейс java.rmi.Remote, а каждый метод должен объявлять исключение java.rmi.RemoteException.

2 Реализация удаленного объекта — класс, реализующий удаленный интерфейс и содержащий бизнес-логику.

3 Stub (заглушка) — прокси-объект на стороне клиента, который выполняет:

— маршалинг (упаковку) параметров;

— установку сетевого соединения;

— отправку запроса;

— ожидание ответа;

— демаршалинг (распаковку) результата.

4 RMI Registry — служба имен, которая позволяет серверу регистрировать свои объекты, а клиенту — находить их по имени.

5 Клиентское приложение — программа, которая ищет удаленный объект в реестре и вызывает его методы.

Синхронный и асинхронный удаленный вызов

Синхронный вызов — классический подход, при котором клиент блокируется до момента получения результата от сервера. Достоинство — простота реализации. Недостаток — клиент не может выполнять другую работу во время ожидания.

Асинхронный вызов — подход, при котором клиент не блокируется. Результат приходит позже через callback-механизм. Достоинство — эффективное использование ресурсов клиента. Недостаток — усложнение архитектуры.

Callback-механизм

Callback (обратный вызов) — это механизм, при котором сервер вызывает метод клиента для передачи результата. Для реализации callback необходимо:

1 Определить удаленный интерфейс callback с методом для получения результата.

2 Клиент должен реализовать этот интерфейс и экспортировать объект.

3 Клиент передает ссылку на свой callback-объект серверу.

4 Сервер, завершив обработку, вызывает callback-метод клиента.

Пул потоков

Пул потоков (Thread Pool) — это коллекция заранее созданных потоков, готовых к выполнению задач. Использование пула потоков позволяет:

— избежать накладных расходов на создание новых потоков;

— ограничить количество одновременно выполняемых задач;

— эффективно управлять ресурсами сервера.

Практическая часть

Формулировка задания:

Клиент передает серверу два целых числа. Сервер вычисляет их наименьшее общее кратное (НОК) и возвращает результат клиенту (синхронно и асинхронно с использованием callback).

Результаты работы:

```
ЗАПУСК КЛИЕНТА #1
==================================================
Подключено к серверу localhost:1099

==================================================
КЛИЕНТ #1
==================================================
Доступные команды:
  calc <число1> <число2>  - синхронный расчет НОК
  async <число1> <число2> - асинхронный расчет НОК
  меню                    - показать это меню
  выход                   - завершить работу
==================================================

Введите команду: calc 13 235

Синхронный вызов...
Результат: НОК(13, 235) = 3055
Время выполнения: 21 мс

Введите команду: async 432 2142

Асинхронный вызов (результат придет через callback)...
```

Рисунок 1 – Результат работы 1-ого клиента

```
Доступные команды:
  calc <число1> <число2>  - синхронный расчет НОК
  async <число1> <число2> - асинхронный расчет НОК
  меню                    - показать это меню
  выход                   - завершить работу
==================================================

Введите команду: calc 3 21

Синхронный вызов...
Результат: НОК(3, 21) = 21
Время выполнения: 4 мс

Введите команду: async 23 43

Асинхронный вызов (результат придет через callback)...
Время отправки: 21 мс
Клиент продолжает работу...
Клиент работает... (1/3)
Клиент работает... (2/3)
Клиент работает... (3/3)

Введите команду:
[CALLBACK] Клиент 'Клиент-1' получил результат: 989
Введите следующую команду (или 'меню' для возврата):
```

Рисунок 2 – Результат работы 2-ого клиента

```
Пул потоков создан. Размер: 10
RMI Registry создан на порту 1099

RMI Сервер LCM запущен
Доступные сервисы:
   - Синхронный:   LCMCalculator
   - Асинхронный:  AsyncLCMCalculator

Ожидание подключений клиентов...
[СИНХРОННЫЙ] Вычисление НОК(13, 235) = 3055 в потоке: RMI TCP Connection(2)-26.199.233.143
[АСИНХРОННЫЙ] Запрос на вычисление НОК(432, 2142) в потоке: RMI TCP Connection(2)-26.199.233.143
[АСИНХРОННЫЙ] Задача отправлена в пул потоков
[АСИНХРОННЫЙ] Результат готов: 51408 в потоке: pool-1-thread-1
[СИНХРОННЫЙ] Вычисление НОК(3, 21) = 21 в потоке: RMI TCP Connection(5)-26.199.233.143
[АСИНХРОННЫЙ] Запрос на вычисление НОК(23, 43) в потоке: RMI TCP Connection(5)-26.199.233.143
[АСИНХРОННЫЙ] Задача отправлена в пул потоков
[АСИНХРОННЫЙ] Результат готов: 989 в потоке: pool-1-thread-2
```

Рисунок 3 – Сервер

Листинг кода:

Server.java:

```
package server;

import interfaces.LCMCalculator;
import interfaces.AsyncLCMCalculator;
import interfaces.LCMCallback;

import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Server implements LCMCalculator, AsyncLCMCalculator {

    private final ExecutorService threadPool;
    private static final int THREAD_POOL_SIZE = 10;

    public Server() {
        this.threadPool = Executors.newFixedThreadPool(THREAD_POOL_SIZE);
        System.out.println("Пул потоков создан. Размер: " + THREAD_POOL_SIZE);
    }

    private int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return Math.abs(a);
    }
```

```
    @Override
    public long calculateLCM(int a, int b) throws RemoteException,
IllegalArgumentException {
        if (a == 0 || b == 0) {
            throw new IllegalArgumentException("Числа не могут быть равны 0");
        }

        long lcm = (long)Math.abs(a) * ((long)Math.abs(b) / gcd(a, b));

        System.out.println("[СИНХРОННЫЙ] Вычисление НОК(" + a + ", " + b + ") = " +
lcm + " в потоке: " + Thread.currentThread().getName());

        return lcm;
    }

    @Override
    public void calculateLCMAsync(int a, int b, LCMCallback callback) throws
RemoteException {
        System.out.println("[АСИНХРОННЫЙ] Запрос на вычисление НОК(" + a + ", " + b +
") в потоке: " + Thread.currentThread().getName());

        threadPool.submit(() -> {
            try {
                Thread.sleep(2000);

                if (a == 0 || b == 0) {
                    callback.onError("Числа не могут быть равны 0");
                    return;
                }

                long result = (long)Math.abs(a) * ((long)Math.abs(b) / gcd(a, b));

                System.out.println("[АСИНХРОННЫЙ] Результат готов: " + result + " в
потоке: " + Thread.currentThread().getName());

                callback.onSuccess(result);

            } catch (InterruptedException e) {
                try {
                    callback.onError("Вычисление было прервано");
                } catch (RemoteException ex) {
                    System.err.println("Ошибка при отправке ошибки клиенту: " +
ex.getMessage());
                }
                Thread.currentThread().interrupt();
            } catch (Exception e) {
                try {
                    callback.onError("Ошибка при вычислении: " + e.getMessage());
                } catch (RemoteException ex) {
                    System.err.println("Ошибка при отправке ошибки клиенту: " +
ex.getMessage());
                }
            }
        });

        System.out.println("[АСИНХРОННЫЙ] Задача отправлена в пул потоков");
    }

    public void shutdown() {
        System.out.println("\nЗавершение работы сервера...");

        threadPool.shutdown();
        try {
            if (!threadPool.awaitTermination(5, TimeUnit.SECONDS)) {
                threadPool.shutdownNow();
            }
        } catch (InterruptedException e) {
```

```
            threadPool.shutdownNow();
        }

        System.out.println("Сервер остановлен");
    }

    public static void main(String[] args) {
        Server server = null;
        try {
            server = new Server();
            final Server finalServer = server;

            Remote stub = UnicastRemoteObject.exportObject(server, 0);

            Registry registry = LocateRegistry.createRegistry(1099);
            System.out.println("RMI Registry создан на порту 1099");

            registry.rebind("LCMCalculator", (LCMCalculator) stub);
            registry.rebind("AsyncLCMCalculator", (AsyncLCMCalculator) stub);

            System.out.println("\nRMI Сервер LCM запущен");
            System.out.println("Доступные сервисы:");
            System.out.println("   - Синхронный:   LCMCalculator");
            System.out.println("   - Асинхронный:  AsyncLCMCalculator");
            System.out.println("\nОжидание подключений клиентов...");

            Runtime.getRuntime().addShutdownHook(new Thread(() -> {
                if (finalServer != null) {
                    System.out.println("\nПолучен сигнал завершения...");
                    finalServer.shutdown();
                }
            }));

        } catch (Exception e) {
            System.err.println("Ошибка сервера: " + e.getMessage());
            e.printStackTrace();
            if (server != null) {
                server.shutdown();
            }
        }
    }
}
```

Client:

```
package client;

import interfaces.LCMCalculator;
import interfaces.AsyncLCMCalculator;
import interfaces.LCMCallback;
import utils.InputValidator;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {

    private final InputValidator validator;
    private final String clientId;
    private LCMCalculator syncService;
    private AsyncLCMCalculator asyncService;
    private LCMCallback callback;

    public Client(String clientId) {
        this.validator = new InputValidator();
        this.clientId = clientId;
```

```
    }

    private boolean connect(String host, int port) {
        try {
            Registry registry = LocateRegistry.getRegistry(host, port);

            syncService = (LCMCalculator) registry.lookup("LCMCalculator");
            asyncService = (AsyncLCMCalculator) registry.lookup("AsyncLCMCalculator");

            callback = new CallbackImpl("Клиент-" + clientId);

            System.out.println("Подключено к серверу " + host + ":" + port);
            return true;

        } catch (Exception e) {
            System.out.println("Ошибка подключения: " + e.getMessage());
            return false;
        }
    }

    private void doSynchronousCall(int a, int b) {
        try {
            System.out.println("\nСинхронный вызов...");
            long start = System.currentTimeMillis();

            long result = syncService.calculateLCM(a, b);

            long time = System.currentTimeMillis() - start;
            System.out.println("Результат: НОК(" + a + ", " + b + ") = " + result);
            System.out.println("Время выполнения: " + time + " мс");

        } catch (Exception e) {
            System.out.println("Ошибка при синхронном вызове: " + e.getMessage());
        }
    }

    private void doAsynchronousCall(int a, int b) {
        try {
            System.out.println("\nАсинхронный вызов (результат придет через
callback)...");
            long start = System.currentTimeMillis();

            asyncService.calculateLCMAsync(a, b, callback);

            long time = System.currentTimeMillis() - start;
            System.out.println("Время отправки: " + time + " мс");
            System.out.println("Клиент продолжает работу...");

            for (int i = 1; i <= 3; i++) {
                Thread.sleep(500);
                System.out.println("Клиент работает... (" + i + "/3)");
            }

        } catch (Exception e) {
            System.out.println("Ошибка при асинхронном вызове: " + e.getMessage());
        }
    }

    private void showMenu() {
        System.out.println("\n" + "=".repeat(50));
        System.out.println("КЛИЕНТ #" + clientId);
        System.out.println("=".repeat(50));
        System.out.println("Доступные команды:");
        System.out.println("  calc <число1> <число2>  - синхронный расчет НОК");
        System.out.println("  async <число1> <число2> - асинхронный расчет НОК");
        System.out.println("  меню                    - показать это меню");
        System.out.println("  выход                   - завершить работу");
```

```
        System.out.println("=".repeat(50));
    }

    public void run(String host, int port) {
        System.out.println("\nЗАПУСК КЛИЕНТА #" + clientId);
        System.out.println("=".repeat(50));

        if (!connect(host, port)) {
            System.out.println("Не удалось подключиться к серверу. Проверьте, запущен
ли сервер.");
            return;
        }

        showMenu();

        java.util.Scanner scanner = new java.util.Scanner(System.in);

        while (true) {
            System.out.print("\nВведите команду: ");
            String input = scanner.nextLine().trim().toLowerCase();

            if (input.equals("выход") || input.equals("exit")) {
                System.out.println("До свидания!");
                break;
            }

            if (input.equals("меню") || input.equals("menu") || input.equals("help"))
{
                showMenu();
                continue;
            }

            if (input.startsWith("calc ")) {
                String[] parts = input.split("\\s+");
                if (parts.length != 3) {
                    System.out.println("Неправильный формат. Используйте: calc
<число1> <число2>");
                    continue;
                }

                try {
                    int a = Integer.parseInt(parts[1]);
                    int b = Integer.parseInt(parts[2]);
                    doSynchronousCall(a, b);
                } catch (NumberFormatException e) {
                    System.out.println("Ошибка: введите целые числа!");
                }

            } else if (input.startsWith("async ")) {
                String[] parts = input.split("\\s+");
                if (parts.length != 3) {
                    System.out.println("Неправильный формат. Используйте: async
<число1> <число2>");
                    continue;
                }

                try {
                    int a = Integer.parseInt(parts[1]);
                    int b = Integer.parseInt(parts[2]);
                    doAsynchronousCall(a, b);
                } catch (NumberFormatException e) {
                    System.out.println("Ошибка: введите целые числа!");
                }

            } else if (!input.isEmpty()) {
                System.out.println("Неизвестная команда. Введите 'меню' для списка
команд.");
```

```
            }
        }

        scanner.close();
        validator.close();
    }

    public static void main(String[] args) {
        String host = "localhost";
        int port = 1099;
        String clientId = "1";

        if (args.length > 0) {
            host = args[0];
        }
        if (args.length > 1) {
            try {
                port = Integer.parseInt(args[1]);
            } catch (NumberFormatException e) {
                System.out.println("Порт должен быть числом. Использую порт 1099");
            }
        }
        if (args.length > 2) {
            clientId = args[2];
        }

        Client client = new Client(clientId);
        client.run(host, port);
    }
}
```

CallbackImpl:

```
package client;

import interfaces.LCMCallback;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CallbackImpl extends UnicastRemoteObject implements LCMCallback {

    private final String clientName;

    protected CallbackImpl(String clientName) throws RemoteException {
        super();
        this.clientName = clientName;
    }

    @Override
    public void onSuccess(long result) throws RemoteException {
        System.out.println("\n[CALLBACK] Клиент '" + clientName + "' получил
результат: " + result);
        System.out.print("Введите следующую команду (или 'меню' для возврата): ");
    }

    @Override
    public void onError(String errorMessage) throws RemoteException {
        System.out.println("\n[CALLBACK] Клиент '" + clientName + "' получил ошибку: "
+ errorMessage);
        System.out.print("Введите следующую команду (или 'меню' для возврата): ");
    }
}
```

AsyncClient:

```
package client;
```

```
import interfaces.AsyncLCMCalculator;
import interfaces.LCMCallback;
import utils.InputValidator;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class AsyncClient {

    public static void main(String[] args) {
        InputValidator validator = new InputValidator();

        try {
            System.out.println("\nАСИНХРОННЫЙ КЛИЕНТ");
            System.out.println("=".repeat(50));

            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            AsyncLCMCalculator service = (AsyncLCMCalculator)
registry.lookup("AsyncLCMCalculator");

            LCMCallback callback = new CallbackImpl("AsyncDemo");

            System.out.println("Подключено к серверу\n");

            while (true) {
                System.out.println("-".repeat(40));
                int a = validator.readInt("Введите первое число: ");
                int b = validator.readInt("Введите второе число: ");

                System.out.println("\nОтправка асинхронного запроса...");
                service.calculateLCMAsync(a, b, callback);

                System.out.println("Запрос отправлен. Ожидание callback...\n");

                for (int i = 1; i <= 5; i++) {
                    Thread.sleep(400);
                    System.out.println("   Работаем... " + i + "/5");
                }

                if (!validator.askContinue()) {
                    break;
                }
            }

        } catch (Exception e) {
            System.err.println("Ошибка: " + e.getMessage());
            e.printStackTrace();
        } finally {
            validator.close();
        }

        System.out.println("Программа завершена");
    }
}
```

AsyncLMCCalculator:

```
package interfaces;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface AsyncLCMCalculator extends Remote {
    void calculateLCMAsync(int a, int b, LCMCallback callback) throws RemoteException;
}
```

LCMCalculator:

```
package interfaces;

import java.rmi.Remote;
import java.rmi.RemoteException;


public interface LCMCalculator extends Remote {
    long calculateLCM(int a, int b) throws RemoteException, IllegalArgumentException;
}
```

LCMCallback:

```
package interfaces;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface LCMCallback extends Remote {
    void onSuccess(long result) throws RemoteException;
    void onError(String errorMessage) throws RemoteException;
}
```

InputValidator:

```
package utils;

import java.util.Scanner;

public class InputValidator {
    private final Scanner scanner;

    public InputValidator() {
        this.scanner = new Scanner(System.in);
    }

    public int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = scanner.nextLine().trim();

            try {
                return Integer.parseInt(input);
            } catch (NumberFormatException e) {
                System.out.println("Ошибка: введите целое число!");
            }
        }
    }

    public int readModeChoice() {
        while (true) {
            System.out.println("\nВыберите режим работы:");
            System.out.println("1. Синхронный вызов");
            System.out.println("2. Асинхронный вызов с callback");
            System.out.println("3. Выход");
            System.out.print("Ваш выбор: ");

            String input = scanner.nextLine().trim();

            try {
                int choice = Integer.parseInt(input);
                if (choice >= 1 && choice <= 3) {
                    return choice;
```

```
                } else {
                    System.out.println("Ошибка: введите число от 1 до 3!");
                }
            } catch (NumberFormatException e) {
                System.out.println("Ошибка: введите число!");
            }
        }
    }

    public boolean askContinue() {
        while (true) {
            System.out.print("\nХотите выполнить еще один расчет? (да/нет): ");
            String input = scanner.nextLine().trim().toLowerCase();

            if (input.equals("да") || input.equals("yes") || input.equals("y")) {
                return true;
            } else if (input.equals("нет") || input.equals("no") || input.equals("n"))
{
                return false;
            } else {
                System.out.println("Пожалуйста, введите 'да' или 'нет'");
            }
        }
    }

    public void close() {
        scanner.close();
    }
}
```